Was wird wo verlangt?

Inventar über alle elf Übungsblätter

Was hier drinsteht

Teil A: Jedes Übungsblatt einzeln – welche Datenstruktur, welche Stream-Operation, welche Art von Lambda dort ausdrücklich gefordert wird.

Teil B: Die Gesamtliste – alles, was über alle elf Blätter zusammen vorkommt, nach Kategorie sortiert. Das ist dein vollständiger Umfang.

Teil C: Was davon realistisch in der Klausur landet und was du weglassen kannst.

Teil A – Blatt für Blatt

Übung 1 – Wiederholung und UML

Kategorie	Was ausdrücklich verlangt wird
Datenstrukturen	`Liste für die 0..*-Beziehungen (Mitarbeiter → Reservierungen, Raum → Reservierungen). Typ nicht vorgegeben, ArrayList reicht.`
Stream / Lambda	keine
Sonstiges	Vererbung Person → Mitarbeiter, Konstruktoren mit super, toString mit festem Ausgabeformat, Assoziationen in beide Richtungen pflegen

Übung 2 – Rekursion

Kategorie	Was ausdrücklich verlangt wird
Datenstrukturen	`int[]-Array (Permutationen). Sonst keine.`
Stream / Lambda	keine
Sonstiges	`Interface Palindrom mit zwei Implementierungen (rekursiv und iterativ), Türme von Hanoi, ggT nach Euklid, Verzeichnis rekursiv mit java.io.File, Laufzeitmessung mit System.nanoTime`

Übung 3 – Innere Klassen

Kategorie	Was ausdrücklich verlangt wird
Datenstrukturen	keine neuen
Stream / Lambda	keine
Innere Klassen	`statische innere Klasse (Conditions in Grain), inneres Interface (Depot.Iterator), private innere Klasse (DepotIterator), innere Klasse mit privatem Konstruktor und immutable (Silo.Status), lokale Klasse (Filter in NumberProcessor), anonyme Klasse (BinaryOperation, vier Stück für die Grundrechenarten)`

> Dieses Blatt deckt alle vier Arten innerer Klassen ab. Genau danach wird im Multiple-Choice-Teil gefragt.

Übung 4 – Generics

Kategorie	Was ausdrücklich verlangt wird
Datenstrukturen	`eigene generische LinkedList<T> mit innerer Node-Klasse und innerem generischem Iterator`
Methoden	addLast, removeFirst, isEmpty, size, get, set, clear, remove, addAll, iterator
Generics	`generische Klasse deklarieren, generische innere Klasse, Bounded Type Parameter (Matrix für numerische Typen), Addition zweier Matrizen unterschiedlichen Typs – dafür Wildcards`
Stream / Lambda	keine

Übung 5 – Comparable, Bubblesort, funktionale Interfaces

Kategorie	Was ausdrücklich verlangt wird
Interfaces	`java.lang.Comparable<T>` für Silo (nach Füllstand) und Harvest (nach Erntejahr); `java.util.Comparator` als Parameter für sort
Funktionale Interfaces	eigenes Interface `MyFunction`; Erweiterung mit zwei default-Methoden (`conditionateInput`, `conditionateOutput`); `Predicate`, `Consumer`
Lambda-Arten	dieselbe Funktion als Lambda, als anonyme Klasse, als Top-Level-Klasse und als echte innere Klasse – zum Vergleich; `odd` als anonyme Klasse, `even` als Lambda
Algorithmus	Bubblesort auf der eigenen LinkedList
Methoden mit Funktion als Parameter	`filter(Predicate)`, `forEach(Consumer)`, `removeIf(Predicate)`, `addIf(T, Predicate)`, `sort(Comparator)`

Übung 6 – Lambdas und Methodenreferenzen

Kategorie	Was ausdrücklich verlangt wird
Funktionale Interfaces	Predicate, Consumer, Function, UnaryOperator – ausdrücklich aus java.util.function
Methodenreferenzen	wo immer möglich statt Lambda; im Review erklären, welche Art verwendet wurde
Lambda-Aufgaben	10 String-Transformationen (Länge über 5 filtern, Großbuchstabe am Anfang, trimmen, in Großbuchstaben, umkehren, auf Länge abbilden, Buchstaben zählen, mit Präfix ausgeben)
Fortgeschritten (Zusatz)	`Function`, die ein `Predicate` zurückgibt (`isDivisorOf(x)`, `isMultipleOf(x)`, `endsWith(x)`); Prädikate kombinieren mit `and`, `or`, `negate`
Datenstrukturen	eigene LinkedList weiterverwenden

> **Das ungewöhnlichste Muster von diesem Blatt – verifiziert**

```
Function<Integer, Predicate<Integer>> isMultipleOf = x -> n -> n % x == 0;
isMultipleOf.apply(5).test(20)      // true

Predicate<Integer> even = n -> n % 2 == 0;
Predicate<Integer> div3 = n -> n % 3 == 0;
even.and(div3).test(12)             // true   -> durch 6 teilbar
even.negate().test(7)               // true   -> ungerade
```

Übung 7 – Collections

Kategorie	Was ausdrücklich verlangt wird
Auswahl mit Begründung	Inventory: schnelles Hinzufügen und Entfernen, effizientes Suchen anhand der Produkt-ID → Map. Zusätzlich Iteration in Einfügereihenfolge → LinkedHashMap
Queue	Producer/Consumer: Entnahme wahlweise nach FIFO oder nach natürlicher Sortierung, per Programmparameter umschaltbar → ArrayDeque gegen PriorityQueue
Map plus Collection	zu jeder Quersumme alle Zeitstempel → `Map<Integer, List<Long>>`; Quersummen auf- und absteigend ausgeben → `TreeMap` mit `descendingKeySet`
Funktionale Interfaces	`filterProducts(Predicate<Product>)`, `applyToProducts(Consumer<Product>)`

Kategorie	Was ausdrücklich verlangt wird
Rückgabetypen	`List<Product> für Suchen, Sortieren, Filtern`
Zusatz	RequestQueue nach FIFO, danach umgestellt auf Priorität → PriorityQueue mit Comparator

TreeMap in beide Richtungen – verifiziert

```
TreeMap<Integer,String> tm = new TreeMap<>(...);
tm.keySet()               ->  [1, 2, 3]        aufsteigend
tm.descendingKeySet()     ->  [3, 2, 1]        absteigend
```

Übung 8 – Streams

Kategorie	Was ausdrücklich verlangt wird
Vulkandaten	`geeignete Collection für Eruption-Objekte (keine Anforderung genannt → ArrayList); 15 Abfragen mit Streams`
Bibliothek – Book	Book so implementieren, dass nach Titel sortiert werden kann → Comparable
Bibliothek – User	Collection für ausgeliehene Bücher: keine Duplikate, sortiert nach Rückgabedatum → TreeSet mit Comparator
Bibliothek – System	`alle Bücher sortiert speichern → TreeSet oder TreeMap; Nutzer schnell über readerID finden → HashMap; verliehene Bücher nach Rückgabedatum → TreeMap`
Stream-Auswertungen	Gesamtseitenzahl, nach Jahr filtern, nach Genre filtern, Durchschnittsbewertung je Genre, drei bestbewertete, Autoren mit den meisten Büchern, nach Seiten sortieren, gefilterte und sortierte Liste mit funktionalen Interfaces als Parameter
Zusatz	`Silo mit PriorityQueue (geringste Haltbarkeit zuerst); Arrays durch Map<GrainType, ...> ersetzen; MusicLibrary mit TreeSet, Song mehrstufig sortierbar nach Künstler, Album, Titel; Top 2 pro Genre; Bereichsabfragen mit subSet, higher, lower; MinHeap implements Queue`

Bereichsabfragen und mehrstufige Sortierung – verifiziert

```
TreeSet<Integer> ts = new TreeSet<>(List.of(10,20,30,40,50));
ts.subSet(20, 45)      ->  [20, 30, 40]
ts.higher(30)          ->  40          das naechste groessere
ts.lower(30)           ->  20          das naechste kleinere
ts.headSet(30)         ->  [10, 20]
ts.tailSet(30)         ->  [30, 40, 50]
ts.descendingSet()     ->  [50, 40, 30, 20, 10]

Comparator.comparing(Song::artist)
          .thenComparing(Song::album)
          .thenComparing(Song::title)
```

Top N pro Gruppe – das anspruchsvollste Muster der Übungen

```
songs.stream().collect(Collectors.groupingBy(Song::genre,
    Collectors.collectingAndThen(Collectors.toList(),
        l -> l.stream().sorted(Comparator.comparingInt(Song::plays).reversed())
              .limit(2).map(Song::title).toList())))

// {Pop=[t0, t3], Rock=[t2, t1]}
```

Übung 9, 10, 11 – C

Blatt	Was ausdrücklich verlangt wird
9	`scanf und printf mit Formatierung, Fibonacci iterativ und rekursiv, Zeitmessung mit clock(), Collatz-Folge. Zusatz: limits.h, call by value nachweisen, Reihen berechnen, Fakultät rekursiv, ggT, Primzahltest`
10	eigene Bibliotheken mit Header-Dateien, eigene Typen mit typedef, Deklaration und Implementierung trennen, geschachtelte Schleifen für Muster
11	`struct Mitarbeiter mit Array und Fehlerbehandlung, multiply mit zwei Pointern, Matrix-Bibliothek über int*-Pointer. Zusatz: Bubblesort in C, verschachtelte structs (Auto mit Motor), eigene String-Funktionen, distance mit Pointer-Rückgabe, dynamische Speicherverwaltung`

Teil B – Die Gesamtliste

Alle Datenstrukturen, die vorkommen

Struktur	Wo verlangt	Warum dort
ArrayList	ÜB 1, 7, 8	Standard, wenn nichts gefordert ist
`eigene LinkedList<T>`	ÜB 4, 5, 6	generische Klasse selbst bauen
HashMap	ÜB 8	Nutzer über readerID finden
LinkedHashMap	ÜB 7	über ID finden UND Einfügereihenfolge
TreeMap	ÜB 7, 8	Quersummen aufsteigend, Bücher nach Rückgabedatum
HashSet	`ÜB 8 (implizit)`	nur Eindeutigkeit
TreeSet	ÜB 8	keine Duplikate UND sortiert
ArrayDeque als Queue	ÜB 7	FIFO-Abarbeitung
PriorityQueue	ÜB 7, 8	nach Priorität, geringste Haltbarkeit zuerst
`Map<K, List<V>>`	ÜB 7	mehrere Werte je Schlüssel
`int[] / int* (C)`	ÜB 2, 11	Arrays und Matrizen

Alle Stream-Operationen, die vorkommen

Kategorie	Operationen
Erzeugen	`liste.stream(), parallelStream(), Files.lines()`
Intermediate	filter, map, mapToInt, distinct, sorted, limit, skip
Terminal – Zahl	count, sum, average, max, min, summaryStatistics, reduce
Terminal – Sammlung	`toList, collect(Collectors.toList), collect(Collectors.toSet), collect(Collectors.joining)`
Terminal – Prüfen	anyMatch, allMatch, noneMatch, findFirst, findAny
Terminal – Ausgabe	forEach
Collectors	counting, summingInt, averagingInt, groupingBy, partitioningBy, joining, toMap, maxBy, minBy, collectingAndThen

Alle Arten von Lambda und funktionalen Interfaces

Art	Beispiel	Wo verlangt
`Predicate<T>`	`s -> s.length() > 5`	ÜB 5, 6, 7, 8

Art	Beispiel	Wo verlangt
`Consumer<T>`	`s -> System.out.println(s)`	ÜB 5, 6, 7
`Function<T,R>`	`s -> s.length()`	ÜB 5, 6, 8
`UnaryOperator<T>`	`s -> s.toUpperCase()`	ÜB 6
`Supplier<T>`	`() -> new ArrayList<>()`	selten, aber MC-relevant
`Comparator<T>`	`Comparator.comparing(Book::getTitle)`	ÜB 5, 7, 8
eigenes Interface	`MyFunction mit int apply(int)`	ÜB 5
default-Methode im Interface	`conditionateInput(Predicate)`	ÜB 5
Function, die Predicate liefert	`x -> n -> n % x == 0`	ÜB 6, Zusatz
Prädikate kombinieren	`even.and(div3), p.negate()`	ÜB 6, Zusatz
anonyme Klasse statt Lambda	`new BinaryOperation() { ... }`	ÜB 3, 5

Alle Arten von Methodenreferenzen

```
String::length              Instanzmethode eines gegebenen Typs
Integer::parseInt           statische Methode
System.out::println         Instanzmethode eines bestimmten Objekts
ArrayList::new              Konstruktor
```

Teil C – Was davon klausurrelevant ist

Priorität	Was	Begründung
PFLICHT	LinkedHashMap, ArrayList, TreeMap, TreeSet, HashMap	die fünf, nach denen in Aufgabentexten gefragt wird
PFLICHT	filter, map, sorted, limit, count, sum, average, toList	kommen in fast jeder Abfrage vor
PFLICHT	groupingBy mit counting und averagingInt	das häufigste Muster überhaupt
PFLICHT	Predicate, Consumer, Comparator als Methodenparameter	in jeder Verwalterklasse
PFLICHT	Comparable mit compareTo	sobald TreeMap oder TreeSet vorkommt
WICHTIG	partitioningBy, joining, anyMatch, max mit Comparator	einzelne Abfragen
WICHTIG	`Map nach Wert sortieren (entrySet, comparingByValue)`	„Top 10" und „nach Häufigkeit"
WICHTIG	Methodenreferenzen unterscheiden können	MC-Frage, kam mehrfach
WICHTIG	PriorityQueue und ArrayDeque	MC-Fragen zu Collections
OPTIONAL	collectingAndThen, maxBy für Top N je Gruppe	genau eine Abfrage in ÜB 8
OPTIONAL	subSet, higher, lower, descendingSet	nur in einer Zusatzaufgabe
OPTIONAL	Function, die ein Predicate zurückgibt	nur Zusatzaufgabe ÜB 6
OPTIONAL	eigene generische LinkedList mit Iterator	ÜB 4, in keiner Klausur aufgetaucht
OPTIONAL	`Wildcards List<? extends K>`	nur eine Teilaufgabe SS 2019

Die Kurzfassung

Von allem, was über elf Übungsblätter verlangt wird, sind die ersten fünf Zeilen der Tabelle etwa achtzig Prozent des Klausurwerts.

Konkret: fünf Datenstrukturen, acht Stream-Operationen, groupingBy, drei funktionale Interfaces als Parameter, und Comparable. Mehr brauchst du für die Programmieraufgaben nicht.

Alles unter OPTIONAL kannst du weglassen, wenn die Zeit knapp ist. Es taucht in den dokumentierten Klausuren gar nicht oder nur in einer einzelnen Teilaufgabe auf.

Teil D – Abgleich mit den echten Klausuren

Was hier passiert

Teil C war eine Einschätzung auf Basis der Übungsblätter. Jetzt der Gegencheck: Was ist in den vier dokumentierten Klausuren tatsächlich vorgekommen?

Datenlage: SS 2019 als vollständige Klausur mit Punkten, SS 2025 als Gedankenprotokoll, SS 2024 und WS 2024/25 als Stichpunkte aus dem Jahrgangschat.

D.1 Was tatsächlich abgefragt wurde

Thema	SS 19	SS 24	WS 24/25	SS 25	Bewertung
Klassenhierarchie nach Vorgabe	ja	ja	ja	ja	in ALLEN vier
Comparable / compareTo	ja	–	–	ja	zweimal, jeweils eigene Teilaufgabe
Verwalterklasse mit Collection	ja	ja	ja	ja	in allen vier
Predicate als Parameter	ja	ja	–	ja	dreimal
Comparator als Parameter	ja	–	–	ja	zweimal
`Streams (filter, map, collect)`	ja	ja	–	ja	dreimal
groupingBy	ja	–	–	ja	zweimal
C: Pointer als Ausgabeparameter	ja	ja	ja	ja	in allen vier
C: struct	ja	–	–	–	einmal
Rekursion + Iteration derselben Aufgabe	ja	–	ja	–	zweimal
Lambda-Syntax als MC-Frage	ja	–	–	ja	zweimal, wörtlich gleich
Methodenreferenzen als MC-Frage	ja	–	–	ja	zweimal, wörtlich gleich
Generics-Instanziierung als MC-Frage	ja	–	–	ja	zweimal, wörtlich gleich
Innere und anonyme Klassen	ja	–	ja	ja	dreimal

D.2 Wo ich meine Einschätzung korrigieren muss

Vier Punkte, die ich in Teil C zu niedrig eingestuft hatte

Wildcards (List<? extends K>). Stand bei mir unter OPTIONAL. Kam aber SS 2019 in Aufgabe 4c mit 3 Punkten dran – wörtlich: „Es sind nur Listen mit Subtypen von K bzw. V zulässig." Bleibt trotzdem niedrige Priorität, weil es genau einmal vorkam und teuer zu lernen ist.

Bubblesort mit Comparator. SS 2019 Aufgabe 3d, 4 Punkte, mit vorgegebenem Pseudocode. Wer stattdessen .sorted(c) schreibt, bekommt sie nicht. Steht in deinem Drillplan als Block E – lass ihn nicht

weg.

parallelStream(). Kam SS 2019 dreimal vor (Aufgaben 3e, 3f, 3g), jeweils ausdrücklich verlangt. Es ist nur ein Wort statt stream(), aber man muss es wissen.

Passiver Iterator. SS 2019 Aufgabe 4e, 3 Punkte. Gemeint ist map.forEach((k,v) -> ...), also interne Iteration – im Gegensatz zum aktiven Iterator mit hasNext und next. Der Begriff steht in keinem Übungsblatt.

Und was ich zu hoch eingestuft hatte

Eigene generische LinkedList mit innerem Iterator. Übungsblatt 4 und 5 bauen das über Wochen auf – in keiner der vier Klausuren kam es vor. Kannst du guten Gewissens weglassen.

PriorityQueue und ArrayDeque. In Übung 7 und 8 ausführlich verlangt, in keiner Klausur als Programmieraufgabe. Nur als Multiple-Choice-Wissen relevant: PriorityQueue liefert das kleinste Element, ArrayDeque ist FIFO.

Bereichsabfragen (subSet, higher, lower). Nur Zusatzaufgabe in Übung 8, nie in einer Klausur. Streichen.

collectingAndThen für Top N je Gruppe. Kam in keiner Klausur vor. Nur mitnehmen, wenn alles andere sitzt.

D.3 Das auffälligste Muster

Die Aufgabe nennt fast immer eine Reihenfolge-Anforderung

Schau dir an, welche Map in den Klausuren verlangt wurde:

```
SS 2019, A3a:  „in der Reihenfolge, in der sie hinzugefuegt wurden"
               ->  LinkedHashMap

SS 2019, A4a:  „nach den Eintraegen sortiert vorliegen"
               ->  TreeMap

SS 2025, A3c:  „die intern eine TreeMap<Point2D, Shape> nutzt"
               ->  TreeMap, direkt vorgegeben
```

Eine schlichte HashMap wurde in keiner der Klausuren verlangt. Es kam immer eine Zusatzbedingung dazu – Einfügereihenfolge oder Sortierung.

Praktische Konsequenz: Wenn du in der Klausur eine Map brauchst und im Text steht irgendetwas zur Reihenfolge, ist HashMap fast sicher die falsche Antwort. Lies die Formulierung genau.

Zweites Muster: Comparable wird nie direkt angekündigt

SS 2019 Aufgabe 4g: „Stellen Sie sicher, dass Objekte der Klasse Translation nach dem String language sortiert werden können." Das Wort Comparable steht nicht da.

SS 2025 Aufgabe 3b: „Point2D Comparable machen" – hier ausnahmsweise doch. Der Grund stand in Teilaufgabe c: dort wurde eine TreeMap mit Point2D als Schlüssel verlangt.

Merkregel: „soll sortiert werden können" oder eine TreeMap/TreeSet mit eigener Klasse bedeutet immer Comparable mit compareTo.

D.4 Die korrigierte Prioritätenliste

Rang	Was	In wie vielen Klausuren
1	Klassenhierarchie: abstrakte Oberklasse, Unterklassen, Verwalter	4 von 4
2	C: rekursive Funktion mit Pointer als Ausgabeparameter	4 von 4
3	Predicate und Comparator als Methodenparameter	3 von 4

Rang	Was	In wie vielen Klausuren
4	Streams: filter, map, sorted, count, collect	3 von 4
5	`Innere und anonyme Klassen (meist als MC oder Fehlersuche)`	3 von 4
6	Comparable mit compareTo	2 von 4, immer als eigene Teilaufgabe
7	groupingBy	2 von 4
8	Die vier wörtlich wiederkehrenden MC-Fragen	2 von 4, identisch
9	Collection mit Reihenfolge-Anforderung wählen und begründen	2 von 4
10	Rekursion und Iteration derselben Aufgabe	2 von 4
11	Bubblesort mit Comparator, parallelStream, passiver Iterator	1 von 4, aber je 3–4 Punkte
12	Wildcards, struct in C	1 von 4
–	eigene LinkedList, PriorityQueue, Bereichsabfragen, collectingAndThen	0 von 4

Was das für deine letzten zwei Tage heißt

Rang 1 und 2 sind in jeder einzelnen Klausur vorgekommen. Wenn du nur zwei Sachen übst, dann diese: das Klassengerüst mit Verwalterklasse, und findMax mit Pointer.

Rang 3 bis 7 sind der Rest der Programmieraufgaben. Rang 8 sind die geschenkten MC-Punkte.

Alles ab Rang 11 nur, wenn Zeit übrig ist. Und die letzte Zeile – vier Themen, die über Wochen in den Übungen aufgebaut wurden und in keiner Klausur auftauchten – kannst du komplett streichen.